

Hand-in exercise 2 - Sudoku OCR

In this hand-in exercise you will use OpenCV's methods to perform optical character recognition (OCR) to detect the digits of a sudoku. The purpose of the hand-in is to show that you are able to combine computer vision algorithms into advanced pipelines.

Your method must take an image of a sudoku as input (see below) and output a grid with the detected digits. You are free to use any OpenCV method you like, however, if you would like to complete the exercise following along with the lectures the suggested method is as follows:

- a. Convert the color image into a binary image using one of the segmentation methods from class
- b. Remove noise in the binary image using morphological operations
- c. Detect either contours or connected components and extract shape features
- d. Train a `cv::NormalBayesClassifier` using the provided training images of numbers 1-9
- e. Make a crop of each of the 9x9 squares (Hint: In this exercise you are allowed to hardcode the crop regions)
- f. For each crop detect if a shape is present, and if there is, use the trained classifier to determine what digit the shape represents
- g. return the sudoku grid as a `std::vector<std::vector<int> >`. Encode 'no digit' as a zero, so the test code below will print the sudoku to the terminal correctly.

Your method should be able to run using the main method below. The cpp-file will also be available on its Learning

```
#include <opencv2/core/core.hpp>
#include <opencv2/highgui/highgui.hpp>
#include <opencv2/imgproc/imgproc.hpp>
#include <iostream>

std::vector<std::vector<int> > sudokuOCR(cv::Mat &image){
    //YOUR CODE HERE
}

std::vector<std::vector<int> > sudokuExampleGrid(){
    //Fill 9x9 grid with arbitrary points.
    //Remove
    std::vector< std::vector< int> > numbers;
    for(int i=0; i<9; i++){
        std::vector<int> row;
```

```

        for(int j=0; j<9; j++){
            int arbitraryNumber = (i+j)%9;
            row.push_back(arbitraryNumber);
        }
        numbers.push_back(row);
    }
    return numbers;
}

void printSudoku(std::vector<std::vector<int> > &numbers){
    std::cout << "Printing sudoku:"<< std::endl;
    std::string line = "-----\n";
    for(int r=0; r<numbers.size(); r++){
        if(r%3==0)
            std::cout << line;
        std::vector<int> row = numbers[r];
        for(int c=0; c<row.size(); c++){
            if(c%3==0)
                std::cout << "| ";
            int n = row[c];
            if( n!=0)
                std::cout << n << " ";
            else
                std::cout << ". ";

        }
        std::cout << "|" <<std::endl;
    }
    std::cout << line << std::endl;
    std::cout << std::endl;
}

/** Main method for running the Sudoku OCR test code.
    Set NO_IMAGE=true to test printing of a 9x9 grid
    Set NO_IMAGE=false to load the sudoku image and
    print the detected numbers in the grid
*/
int main(int argc, char* argv[])
{
    bool NO_IMAGE = false;

    std::vector<std::vector<int> > numbers;
    if(NO_IMAGE){
        numbers = sudokuExampleGrid();
    }else{
        if(argc != 2){
            std::cout << "Usage: ./main <imageFile>" << std::endl;
            return -1;
        }
    }
}

```

```

        cv::Mat src = cv::imread(argv[1], cv::IMREAD_COLOR);
        cv::namedWindow("Input image", cv::WINDOW_FULLSCREEN);
        cv::imshow("Input image", src);
        cv::waitKey(0);

        numbers = sudokuOCR(src);
    }

    printSudoku(numbers);
}

```

The following is the image to be used in the exercise. It will be provided as a png-file on itsLearning together with training images for the numbers 1-9.

		5		7	8			1
				1				
6				3			2	
				8			6	
5	3	2	7			8		9
4					5			
				4			1	3
		1	6			2	8	
3				2		5		